Groupy

Rodrigo Perestrelo, Lennard Last
ID2201 Distributed Systems, Basic Course

September 30, 2026

1 Introduction

The objective of this assignment is to design, implement, and empirically evaluate a group membership service that guarantees atomic multicast within view synchrony. The service coordinates the state of multiple distributed application-layer processes, ensuring they execute an identical sequence of state changes (represented visually through cyclic RGB color rotations).

2 Iterative Architectural Progression

We developed the system through three distinct iterations, progressively addressing failure detection and atomic delivery across node crashes.

2.1 Iteration 1: Centralized Multicast with Dynamic Joins (GMS1)

The initial architecture, `gms1`, provides basic group membership and multicast serialization under the assumption of zero crashes. A single node is designated as the group leader, while all other participants act as slaves:

- **Sequenced Multicast:** All application state changes are submitted via `{mcast, Msg}`. Slaves forward requests to the leader, which serializes them through `bcast/3` across the group, enforcing total ordering.
- **Deterministic Admission:** New nodes contact an existing peer with `{join, Wrk, Peer}`. The request reaches the leader, which appends the new peer to the tail of `Slaves` and `Group`, multicasting an updated view.
- **State Catch-Up on Admission:** When admitted, a joining node multicasts a `{state_request, Ref}`. Because all multicasts are totally ordered, existing nodes observe this request at the same logical index and reply with their current state (`{state, Ref, Color}`), allowing the new node to synchronize before applying deferred pipeline changes.

The Fatal Flaw: The leader represents an unmitigated single point of failure. If the leader halts, slaves block permanently in their forwarding loop.

2.2 Iteration 2: Failure Detection and Leader Election (GMS2)

To eliminate deadlocks on leader crash, `gms2` introduces asynchronous failure detection and autonomous succession:

- **Targeted Process Monitors:** Slaves monitor exclusively the current leader via `erlang:monitor(proces`
  `Leader)`.

- **Deterministic Election:** Upon receiving a `'DOWN'` signal, slaves enter `election/4`. Because all nodes preserve identical list ordering, the first element (`[Self | Rest]`) autonomously assumes the leadership role and broadcasts the updated view. Remaining peers redirect their monitor to the new leader.
- **Join Timeouts:** A `?timeout` clause prevents joining nodes from hanging indefinitely if a leader crash coincides with a join request.

```
election(Id, Master, Slaves, Group) ->
    Self = self(),
    case Slaves of
        [Self | Rest] ->
            bcast(Id, {view, Slaves, Group}, Rest),
            Master ! {view, Group},
            leader(Id, Master, Rest, Group);
        [Leader | Rest] ->
            erlang:monitor(process, Leader),
            slave(Id, Master, Leader, Rest, Group)
    end.
```

2.3 Iteration 3: Atomic Multicast with View Synchrony (GMS3)

While `gms2` survives clean exits, it violates atomic delivery when a leader crashes mid-broadcast. `gms3` enforces view synchrony by introducing message sequence numbers N, caching the `Last` delivered broadcast, and leveraging two structural invariants:

1. **Sequential Dispatch Invariant:** The leader transmits messages strictly in the order of the `Slaves` list.
2. **Succession Invariant:** The first slave in `Slaves` is always elected as the next leader.

Deduction: If the dead leader delivered a message to *any* node in the group before crashing, the designated successor (`Self`) is guaranteed to have received it. Upon election, the new leader resends `Last` to all surviving peers before broadcasting the newly installed view:

```
election(Id, Master, N, Last, Slaves, Group) ->
    Self = self(),
    case Slaves of
        [Self | Rest] ->
            bcast(Id, Last, Rest),
            bcast(Id, {view, N, Slaves, Group}, Rest),
            Master ! {view, Group},
            leader(Id, Master, N + 1, Rest, Group);
        [NewLeader | Rest] ->
            erlang:monitor(process, NewLeader),
            slave(Id, Master, NewLeader, N, Last, Rest, Group)
    end.
```

Duplicate Suppression: To prevent peers that already received `Last` prior to the crash from processing it twice (which would rotate state out of phase), slaves filter out duplicate sequence numbers ($I < N$):

```
{msg, I, _} when I < N ->
    slave(Id, Master, Leader, N, Last, Slaves, Group);
{msg, N, Msg} ->
    Master ! Msg,
    Last2 = {msg, N, Msg},
    slave(Id, Master, Leader, N + 1, Last2, Slaves, Group);
{view, I, _, _} when I < N ->
    slave(Id, Master, Leader, N, Last, Slaves, Group);
```

```
{view, N, [Leader | Slaves2], Group2} ->
    Master ! {view, Group2},
    Last2 = {view, N, [Leader | Slaves2], Group2},
    slave(Id, Master, Leader, N + 1, Last2, Slaves2, Group2);
```

3 Empirical Evaluation

3.1 Evaluation 1: GMS1 Failure-Free Coordination and Leader Deadlock

We deployed an initial 3-node cluster under `gms1` with a cycle interval of 1000 ms to evaluate behavior under both static coordination and leader termination:

```
1> c(gui), c(worker), c(test), c(gms1).
{ok,gms1}
2> W1 = test:first(1, gms1, 1000).
<0.92.0>
3> W2 = test:add(2, gms1, W1, 1000).
<0.95.0>
4> W3 = test:add(3, gms1, W1, 1000).
<0.98.0>
5> exit(W1, kill).
true
```

Observed Result:

- **Failure-Free Operation:** All three nodes updated their states simultaneously, confirming that centralized sequencer routing over FIFO links satisfies total ordering when nodes do not crash.
- **Leader Crash Deadlock:** Upon executing `exit(W1, kill)`, surviving workers `W2` and `W3` completely stopped updating and froze permanently. Because `gms1` lacks process monitors and election mechanisms, surviving slaves continued forwarding state requests to the deceased leader PID without error recovery, empirically demonstrating the single-point-of-failure limitation.

3.2 Evaluation 2: GMS2 State Divergence Under Crash Injection

We initialized a 4-node cluster under `gms2` with crash injection probability arghh = 150:

```
1> c(gms2).
{ok,gms2}
2> W1 = test:first(1, gms2, 500).
3> W2 = test:add(2, gms2, W1, 500).
4> W3 = test:add(3, gms2, W1, 500).
5> W4 = test:add(4, gms2, W1, 500).
leader 1:  crash
```

Observed Result: Worker 1 crashed while multicasting a color update. Worker 2 received the message, updated its color, and promoted itself to leader upon receiving the monitor signal. However, Workers 3 and 4 missed the message prior to the crash. Consequently, Worker 2 displayed a color state completely out of phase with Workers 3 and 4. As new updates were broadcast, this offset persisted permanently, confirming that `gms2` fails to preserve consistency under mid-broadcast failure.

3.3 Evaluation 3: GMS3 View Synchrony and Self-Healing

We repeated the crash experiment using `gms3`:

```
1> c(gms3).
{ok,gms3}
2> W1 = test:first(1, gms3, 500).
3> W2 = test:add(2, gms3, W1, 500).
4> W3 = test:add(3, gms3, W1, 500).
5> W4 = test:add(4, gms3, W1, 500).
leader 1:  crash
```

Observed Result: Worker 1 crashed mid-broadcast. Worker 2 detected the crash, retransmitted `Last` to Workers 3 and 4, and broadcast the new view. Workers 3 and 4 caught up on the missing transition, while any duplicate on Worker 2 was dropped by the $I < N$ guard. All surviving windows continued rotating colors in lockstep.

3.4 Evaluation 4: The Rolling Group Experiment

To answer the core question of Section 3.4 ("*Can we keep a group rolling by adding more nodes as existing nodes die?*"), we subjected `gms3` to continuous leader crashes while dynamically adding new nodes:

```
leader 1:  crash
6> W5 = test:add(5, gms3, W2, 500).
leader 2:  crash
7> W6 = test:add(6, gms3, W3, 500).
```

Observed Result: The group remained operational indefinitely. Whenever a leader crashed, the next sequential slave assumed leadership without state divergence. Newly admitted workers requested the current color, caught up with in-flight messages, and seamlessly merged into the rolling view.

4 Optional Task for Extra Bonus: What Could Possibly Go Wrong

Erlang only guarantees that messages between two processes arrive in FIFO order, not that they arrive at all. Our implementations `gms1`–`gms3` silently rely on reliable delivery. If a `{msg, N, Msg}` is lost, the slave never delivers message N, and all later messages stay unmatched in its mailbox, so the slave stalls. Lost views, join requests or acknowledgments cause similar problems. To handle this, we extended `gms3` to `gms4` with acknowledgments and retransmission, in the same way TCP builds reliable delivery on top of an unreliable channel.

4.1 Implementation

On the slave side, an acknowledgment is sent back to the leader whenever a message or view is received. Duplicates (`I < N`) are acknowledged again, because a resend usually means that the earlier ack was lost:

```
{msg, I, _} when I < N ->
    send_ack(Leader, I),
    slave(Id, Master, Leader, N, Last, Slaves, Group);
{msg, N, Msg} ->
    Master ! Msg,
    send_ack(Leader, N),
    ...
```

```
{view, I, _, _} when I < N ->
    send_ack(Leader, I),
    slave(Id, Master, Leader, N, Last, Slaves, Group);
{view, N, [Leader|Slaves2], Group2} ->
    Master ! {view, Group2},
    send_ack(Leader, N),
    ...
```

The leader stores a `MsgBuffer` containing every message and view it has sent, and a list `Acks` with the highest acknowledged sequence number of each slave. Every time an ack arrives, the entry of that slave is updated (never decreased) and the buffer is pruned: a message is removed once all current slaves have acknowledged it, i.e. when its number is at most the minimum of the acks.

Whenever a defined `resend_interval` has elapsed, the leader receives a `resend` message from a timer started with `erlang:send_after/3`. Upon a `resend`, the `MsgBuffer` is checked against `Acks`, and every buffered message with a number above the last ack of a slave is sent to that slave again. Since the slaves already filter duplicates with the `I < N` clauses of `gms3`, retransmissions are idempotent.

Two details concern joins and elections. A joining slave is registered with ack $N-1$, where N is the number of its first view, so that the view stays in the buffer until the joiner acknowledges it. A newly elected leader starts its own resend timer and initializes its buffer with `Last` and the new view, so that they can be resent if the election broadcast was lost.

To test the mechanism, all broadcasts, resends and acks go through a send function that drops each message with probability 1/100. Each dropped message and resent is logged to the terminal:

```
1> test:more(5, gms4, 1000).
node <0.94.0>:  dropped a msg message
<0.87.0>
leader 1:  resending 8 message(s) to <0.94.0>
node <0.96.0>:  dropped a ack message
node <0.92.0>:  dropped a msg message
leader 1:  resending 1 message(s) to <0.92.0>
node <0.96.0>:  dropped a ack message
leader 1:  resending 1 message(s) to <0.92.0>
```

The output shows: whenever a message is not acknowledged because the initial dispatch of the message or the subsequent ack failed, it is resent.

4.2 Performance Impact

- **Message overhead:** Every multicast causes one additional ack per slave. Since an ack covers all lower numbers, acks could be batched or sent cumulatively to reduce this.
- **Latency:** A lost message is delayed by up to one resend interval plus the retransmission. A short interval recovers quickly but causes needless duplicates when the round trip is longer than the interval, while a long one makes recovery slow.
- **Memory:** The leader's buffer size depends on the slowest slave, and a slave that is alive but stuck makes it grow.
- **Throughput:** The leader already serialises all multicasts, and it now also handles acks, buffer pruning and retransmissions, which increases its load.

5 Conclusions

This laboratory demonstrated the design of an atomic multicast protocol operating under view synchrony. Centralized FIFO multicasting (`gms1`) provides total ordering in static topologies but deadlocks upon leader crash. Adding naive failure detection (`gms2`) enables failover but violates atomic delivery during mid-transmission failures. By combining deterministic list ordering, sequential succession, retransmission of the last delivered message, and monotonic sequence filtering, `gms3` guarantees that all correct processes agree on an identical state history across dynamic views.